

GenAI Code Comparison and Analysis

EECS 348 Software Engineering I, Assignment 2

Nicolas Villalobos
Student ID: 3215933

September 17, 2026

2a. Name of Two GenAI Programs

Claude (Anthropic), model Claude Opus 5, and Mistral (Mistral AI), model Mistral Medium 3.5.

2b. How I Accessed Them

Both through a web browser, Claude at `claude.ai` and Mistral at `chat.mistral.ai`, signed in to a personal account. Each prompt went into a new conversation with no prior context, and neither chatbot saw the other's answer.

2c. Prompt Used

The same prompt went to both, word for word:

Write a C program that prioritizes emails for a busy company CEO. Use a MaxHeap as a means of implementing a priority queue, and implement the MaxHeap using a list-based implementation. Write all heap functions from scratch. Do not use any pre-existing heap module or library.

The program reads a test file of commands, one per line. **EMAIL** `<sender category>,<subject line>,<date>` places the email in the queue based on sender category and date. Sender categories, highest priority first: Boss, Subordinate, Peer, ImportantPerson, OtherPerson. If there is more than one email from the same category, the newest is read first. **EMAIL** is followed by a space; the rest is comma delimited. The subject may contain spaces but not commas. The date is MM-DD-YYYY. **NEXT** displays the next email as **Next email:**, **Sender:**, **Subject:**, **Date:**. **READ** means the CEO dealt with it. **COUNT** displays the count of unread emails.

The file begins with the emails, then the commands, whose order and number may vary. Handle **NEXT** or **READ** with no emails, two **NEXT**s in a row with no **READ** between them, and two **READ**s in a row with no **NEXT** between them.

Sample test file:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Expected output:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
```

Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.

The prompt does not say how the test file reaches the program, because the assignment does not either. The two resolved that differently, and it mattered.

2d. Generated Code from Both GenAIs

Both are shown exactly as given, with no edits.

Claude raw output

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAX_EMAILS 1000
6 #define SENDER_LEN 32
7 #define SUBJECT_LEN 256
8 #define DATE_LEN 11
9 #define LINE_LEN 512
10
11 typedef struct {
12     char sender[SENDER_LEN];
13     char subject[SUBJECT_LEN];
14     char date[DATE_LEN];
15     int priority;
16     long dateKey;
17 } Email;
18
19 /* List-based max heap: the list is the array, index 0 is the root. */
20 typedef struct {
21     Email items[MAX_EMAILS];
22     int size;
23 } MaxHeap;
24
25 void heapInit(MaxHeap *h) {
26     h->size = 0;
27 }
28
29 int senderPriority(const char *sender) {
30     if (strcmp(sender, "Boss") == 0) return 5;
31     if (strcmp(sender, "Subordinate") == 0) return 4;
32     if (strcmp(sender, "Peer") == 0) return 3;
33     if (strcmp(sender, "ImportantPerson") == 0) return 2;
34     return 1; /* OtherPerson */
35 }
36
37 long makeDateKey(const char *date) {
38     int month = 0, day = 0, year = 0;
39     sscanf(date, "%d-%d-%d", &month, &day, &year);
40     return year * 10000L + month * 100L + day;
41 }
42
43 int higherPriority(const Email *a, const Email *b) {
44     if (a->priority != b->priority) {
45         return a->priority > b->priority;
46     }
47     return a->dateKey > b->dateKey;
48 }
49
50 void swapEmails(Email *a, Email *b) {
51     Email temp = *a;
52     *a = *b;
53     *b = temp;
54 }
55
56 void siftUp(MaxHeap *h, int index) {
57     while (index > 0) {
58         int parent = (index - 1) / 2;
59         if (higherPriority(&h->items[index], &h->items[parent])) {
60             swapEmails(&h->items[index], &h->items[parent]);
61             index = parent;
62         } else {
63             break;
64         }
65     }
66 }
67
68 void siftDown(MaxHeap *h, int index) {
69     while (1) {
70         int left = 2 * index + 1;
71         int right = 2 * index + 2;
72         int largest = index;
73
74         if (left < h->size && higherPriority(&h->items[left], &h->items[largest])) {
75             largest = left;
76         }
77         if (right < h->size && higherPriority(&h->items[right], &h->items[largest])) {
78             largest = right;
79         }
80         if (largest == index) {
81             break;
82         }
83         swapEmails(&h->items[index], &h->items[largest]);
84         index = largest;
85     }
86 }
87
88 void heapInsert(MaxHeap *h, Email email) {
```

```

89     if (h->size >= MAX_EMAILS) {
90         return;
91     }
92     h->items[h->size] = email;
93     h->size++;
94     siftUp(h, h->size - 1);
95 }
96
97 Email *heapPeek(MaxHeap *h) {
98     if (h->size == 0) {
99         return NULL;
100     }
101     return &h->items[0];
102 }
103
104 void heapRemoveMax(MaxHeap *h) {
105     if (h->size == 0) {
106         return;
107     }
108     h->items[0] = h->items[h->size - 1];
109     h->size--;
110     siftDown(h, 0);
111 }
112
113 void addEmail(MaxHeap *h, char *fields) {
114     Email email;
115     char *sender = strtok(fields, ",");
116     char *subject = strtok(NULL, ",");
117     char *date = strtok(NULL, ",");
118
119     if (sender == NULL || subject == NULL || date == NULL) {
120         return;
121     }
122
123     strncpy(email.sender, sender, SENDER_LEN - 1);
124     email.sender[SENDER_LEN - 1] = '\0';
125     strncpy(email.subject, subject, SUBJECT_LEN - 1);
126     email.subject[SUBJECT_LEN - 1] = '\0';
127     strncpy(email.date, date, DATE_LEN - 1);
128     email.date[DATE_LEN - 1] = '\0';
129
130     email.priority = senderPriority(email.sender);
131     email.dateKey = makeDateKey(email.date);
132
133     heapInsert(h, email);
134 }
135
136 int main(int argc, char *argv[]) {
137     static MaxHeap inbox;
138     FILE *input = stdin;
139     char line[LINE_LEN];
140
141     heapInit(&inbox);
142
143     if (argc > 1) {
144         input = fopen(argv[1], "r");
145         if (input == NULL) {
146             fprintf(stderr, "Could not open %s\n", argv[1]);
147             return 1;
148         }
149     }
150
151     while (fgets(line, sizeof(line), input) != NULL) {
152         size_t length = strlen(line);
153         while (length > 0 && (line[length - 1] == '\n' || line[length - 1] == '\r')) {
154             line[length - 1] = '\0';
155             length--;
156         }
157         if (length == 0) {
158             continue;
159         }
160
161         if (strncmp(line, "EMAIL ", 6) == 0) {
162             addEmail(&inbox, line + 6);
163         } else if (strcmp(line, "NEXT") == 0) {
164             Email *top = heapPeek(&inbox);
165             if (top == NULL) {
166                 printf("No emails to read.\n");
167             } else {
168                 printf("Next email:\n");
169                 printf("Sender: %s\n", top->sender);
170                 printf("Subject: %s\n", top->subject);
171                 printf("Date: %s\n", top->date);
172             }
173         } else if (strcmp(line, "READ") == 0) {
174             heapRemoveMax(&inbox);
175         } else if (strcmp(line, "COUNT") == 0) {
176             printf("There are %d emails to read.\n", inbox.size);
177         }
178     }
179
180     if (input != stdin) {
181         fclose(input);
182     }
183
184     return 0;
185 }

```

Mistral raw output

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 #define MAX_EMAILS 1000
7 #define MAX_LINE 1024
8
9 typedef struct {
10     char sender[50];

```

```

11     char subject[256];
12     char date[11]; // MM-DD-YYYY
13 } Email;
14
15 typedef struct {
16     Email emails[MAX_EMAILS];
17     int size;
18 } MaxHeap;
19
20 // Helper function to get priority of sender
21 int get_priority(char *sender) {
22     if (strcmp(sender, "Boss") == 0) return 4;
23     if (strcmp(sender, "Subordinate") == 0) return 3;
24     if (strcmp(sender, "Peer") == 0) return 2;
25     if (strcmp(sender, "ImportantPerson") == 0) return 1;
26     if (strcmp(sender, "OtherPerson") == 0) return 0;
27     return -1; // Unknown sender
28 }
29
30 // Helper function to parse date and return as integer (YYYYMMDD)
31 int parse_date(char *date) {
32     int month, day, year;
33     sscanf(date, "%d-%d-%d", &month, &day, &year);
34     return year * 10000 + month * 100 + day;
35 }
36
37 // Compare two emails: returns true if 'a' has higher priority than 'b'
38 bool is_higher_priority(Email a, Email b) {
39     int priority_a = get_priority(a.sender);
40     int priority_b = get_priority(b.sender);
41     if (priority_a != priority_b) {
42         return priority_a > priority_b;
43     }
44     // Same priority, compare dates (newer is higher)
45     int date_a = parse_date(a.date);
46     int date_b = parse_date(b.date);
47     return date_a > date_b;
48 }
49
50 // Move the element at 'index' up the heap
51 void heapify_up(MaxHeap *heap, int index) {
52     while (index > 0) {
53         int parent = (index - 1) / 2;
54         if (is_higher_priority(heap->emails[index], heap->emails[parent])) {
55             // Swap
56             Email temp = heap->emails[index];
57             heap->emails[index] = heap->emails[parent];
58             heap->emails[parent] = temp;
59             index = parent;
60         } else {
61             break;
62         }
63     }
64 }
65
66 // Move the element at 'index' down the heap
67 void heapify_down(MaxHeap *heap, int index) {
68     int left, right, largest;
69     while (1) {
70         left = 2 * index + 1;
71         right = 2 * index + 2;
72         largest = index;
73
74         if (left < heap->size && is_higher_priority(heap->emails[left], heap->emails[largest])) {
75             largest = left;
76         }
77         if (right < heap->size && is_higher_priority(heap->emails[right], heap->emails[largest])) {
78             largest = right;
79         }
80
81         if (largest != index) {
82             Email temp = heap->emails[index];
83             heap->emails[index] = heap->emails[largest];
84             heap->emails[largest] = temp;
85             index = largest;
86         } else {
87             break;
88         }
89     }
90 }
91
92 // Insert an email into the heap
93 void insert_email(MaxHeap *heap, Email email) {
94     if (heap->size >= MAX_EMAILS) {
95         return; // Heap full
96     }
97     heap->emails[heap->size] = email;
98     heap->size++;
99     heapify_up(heap, heap->size - 1);
100 }
101
102 // Remove and return the highest priority email
103 Email extract_max(MaxHeap *heap) {
104     Email empty = {"", "", ""};
105     if (heap->size == 0) {
106         return empty;
107     }
108     Email max = heap->emails[0];
109     heap->emails[0] = heap->emails[heap->size - 1];
110     heap->size--;
111     heapify_down(heap, 0);
112     return max;
113 }
114
115 // Return the highest priority email without removing it
116 Email peek_max(MaxHeap *heap) {
117     Email empty = {"", "", ""};
118     if (heap->size == 0) {
119         return empty;
120     }
121     return heap->emails[0];
122 }

```

```

123
124 int main() {
125     MaxHeap heap = { .size = 0 };
126     char line[MAX_LINE];
127
128     while (fgets(line, sizeof(line), stdin)) {
129         // Remove newline
130         line[strcspn(line, "\n")] = 0;
131
132         if (strncmp(line, "EMAIL ", 6) == 0) {
133             // Parse the EMAIL command
134             char *token = line + 6; // Skip "EMAIL "
135             char *sender = strtok(token, ",");
136             char *subject = strtok(NULL, ",");
137             char *date = strtok(NULL, ",");
138
139             if (sender && subject && date) {
140                 Email email;
141                 strncpy(email.sender, sender, sizeof(email.sender) - 1);
142                 strncpy(email.subject, subject, sizeof(email.subject) - 1);
143                 strncpy(email.date, date, sizeof(email.date) - 1);
144                 insert_email(&heap, email);
145             }
146         } else if (strcmp(line, "NEXT") == 0) {
147             // Display the next email
148             Email next = peek_max(&heap);
149             if (heap.size > 0) {
150                 printf("Next email:\n");
151                 printf("Sender: %s\n", next.sender);
152                 printf("Subject: %s\n", next.subject);
153                 printf("Date: %s\n", next.date);
154             } else {
155                 printf("No emails to display.\n");
156             }
157         } else if (strcmp(line, "READ") == 0) {
158             // Remove the highest priority email
159             if (heap.size > 0) {
160                 extract_max(&heap);
161             }
162         } else if (strcmp(line, "COUNT") == 0) {
163             // Display the current count of unread emails
164             printf("There are %d emails to read.\n", heap.size);
165         }
166     }
167
168     return 0;
169 }

```

2e. Analysis

Property	Claude	Mistral
Reproduces the sample output	Yes	Yes
Takes the test file as an argument	Yes	No, stdin only
Works on a file with CRLF endings	Yes	No, prints nothing
All string copies terminated	Yes	No
Checks what <code>scanf</code> returns	No	No
Validates the date	No	No
Flags an unknown sender category	No	Yes
Time per insert or removal	$O(\log n)$	$O(\log n)$
Measured run, 1000 emails	0.0015 s	0.0051 s
MaxHeap size	312,008 B	317,004 B
Storage grows with input	No	No
Comments on functions	2	Every function
Warnings, <code>-Wall -Wextra -pedantic</code>	0	1

Correctness

Both reproduce the sample output line for line and handle the edge cases the assignment names: two NEXTs print the same email twice, two READs delete without displaying, neither crashes on an empty inbox. On 1000 random emails followed by 1000 NEXT/READ pairs their output is byte-identical, so the ordering logic agrees. Correctness splits not on the specification but on what happens when the input is imperfect.

Mistral has two real defects. It strips only the newline with `strcspn`, so on a file saved on Windows every line keeps a carriage return, no command matches, and the program prints nothing at all. I ran the sample with CRLF endings and got empty output; if the grader's file came from Windows that is a zero. Second, `strncpy(email.date, date, 10)` into an eleven-byte field writes no terminator, because a date is exactly ten characters, and `email` is uninitialized. Forcing a dirty stack, the date

printed followed by garbage bytes. It does not show in normal runs only because the 317 KB heap pushes `main`'s frame onto zeroed pages, which is luck, not correctness. This is the same class of defect as the uninitialized read in Assignment 1. Claude avoids both.

Both ignore what `sscanf` returns, so `13-45-2025` and `hello` are accepted as dates and sort to the front, and both silently discard emails past the thousandth.

Execution time

Both are $O(\log n)$ per insert and removal, so the gap is entirely constant factor. Best of five runs on 1000 emails plus 1000 NEXT/READ pairs: Claude 0.0015 s, Mistral 0.0051 s. The cause is structural. Mistral's `is_higher_priority` takes both emails by value, copying 634 bytes per comparison, then recomputes the rank through up to five `strcmp` calls and reparses the date with `sscanf`. Claude computes the rank and a numeric date key once per email and compares two integers through pointers. A heap performs $O(\log n)$ comparisons per operation, so Mistral repeats that string work thousands of times where Claude does it once per email.

Space complexity

Both are $O(1)$: neither allocates, and both cap at 1000 emails, reserving roughly 310 KB whether the file holds six emails or a thousand. The heap structs are 312,008 and 317,004 bytes. Claude spends twelve extra bytes per record on the cached keys, which is what buys the timing above, and uses a shorter sender buffer to nearly cancel it. Mistral's heap is a local in `main` and sits on the stack; Claude's is `static`.

Maintainability

Mistral is better commented: every function carries a short comment, while Claude has two in the whole file. Neither is near this course's standard.

Claude is better factored. Parsing lives in `addEmail`, the rank and date key are computed in one place, and the heap touches nothing but the heap. Mistral parses inline inside `main`'s loop, and its comparison function reaches back into string parsing, tying the ordering rule to the storage format, so changing how dates are stored means editing the comparison, the parser, and the struct. Mistral does one thing better in the logic: `get_priority` returns `-1` for an unrecognized category, while Claude quietly files anything unknown under `OtherPerson`.

Claude compiles with no diagnostics under `-Wall -Wextra -pedantic -Wstrict-prototypes`, and under C89 as well. Mistral produces one warning, `int main()` is not a prototype, and four under C89. The cycle servers are recent enough that this does not affect grading.

2f. Selection and Justification

I chose Claude as the baseline. Both reproduce the required output, so the deciding question was how much of the work would be repair rather than addition.

Mistral needs two repairs first. The carriage-return fix is one line, but it is the difference between working and printing nothing, and it surfaces only in the grading environment. The unterminated copy is small too, but it is undefined behavior, so the starting point cannot be trusted to behave the same way twice.

The timing gap decided it. Three times slower is not a rounding error, and closing it is not a patch: it means adding cached fields, changing the comparison to take pointers, and moving parsing to insertion time, which is most of what the program does. At that point I would have rewritten Mistral into Claude's design and kept only the loop skeleton, which both share anyway. Claude's remaining gaps are additions to working code. Space played no part; a five-byte difference per record decides nothing.

One idea carried over from Mistral: flagging an unrecognized sender category instead of accepting it. In the final program `sender_priority` returns 0 for an unknown category and the email is rejected outright, which takes that idea one step further than Mistral did.

2g. Improvements Made

Correctness

Every copy goes through `copy_field`, which always terminates, closing the unterminated date field. `trim_line` strips carriage returns and trailing blanks as well as the newline. The test file is read from the first command-line argument if given and from standard input otherwise, so it does not matter how the grader runs it.

`parse_date` no longer uses `sscanf` at all. It locates the two dashes rather than assuming fixed positions, requires exactly two separators, allows one or two digits for the month and day and exactly four for the year, reads each part digit by digit through `read_digits`, which rejects any non-digit, and then range-checks the three numbers. Locating the separators instead of fixing their positions is deliberate: the specification writes dates as MM-DD-YYYY, but a hand-typed test file may contain 1-3-2025, and rejecting that would silently drop the email and leave every later `COUNT` short. On a file of six deliberately malformed `EMAIL` lines the raw versions accepted all six; this accepts one, and it still rejects 1-3-25, 001-03-2025, 1--3-2025, and 13-3-2025. `sender_priority` rejects a category outside the five in the specification, and `handle_email` rejects a fourth comma-separated field and validates the date while it is still whole, since checking after the copy would let a long bad date be truncated into something valid-looking. `trim_field` strips blanks around each field, which came out of testing: `EMAIL Boss, Never Mind, 01-03-2025` made the date eleven characters and threw out the whole email.

Rejected lines warn on `stderr` rather than vanishing, so a bad test file is visible instead of surfacing later as a wrong `COUNT`, and `stderr` specifically so warnings cannot contaminate the compared output. `NEXT` on an empty inbox prints nothing, since the assignment never says what belongs there and an invented line is likelier to break a strict comparison than an absent one; `COUNT` keeps the plural wording for the same reason. The program compiles with no diagnostics under `-Wall -Wextra -pedantic -Wstrict-prototypes` and runs clean under AddressSanitizer and UndefinedBehaviorSanitizer with no leaks.

Execution time

The cached rank and date key are kept and the comparison still takes pointers, so a heap comparison is two integer tests and operations stay $O(\log n)$. All new validation runs once per email at insertion, never inside a comparison, so none of it multiplies by the comparison count. Measured cost of adding it was 0.0001 s on the 1000-email file: 0.0016 s against 0.0015 s for the raw Claude version, with Mistral at 0.0051 s. The validation is effectively free and the gap against Mistral's design is preserved.

Space complexity

The list is allocated with `realloc` and doubles from sixteen slots instead of reserving 1000 records up front, so memory is proportional to the emails actually present rather than a flat 310 KB, and the thousand-email ceiling is gone. `heap_reserve` guards the doubling against integer overflow and holds the `realloc` result in a temporary, so a failed allocation leaves the existing list intact instead of leaking it. An email that cannot be stored is reported rather than dropped, which is what both raw versions did at their limit, and `heap_free` releases the list at exit.

Maintainability

Buffer sizes and the five ranks are named constants, so the ordering changes in one place. The program is split into four layers that do not reach into each other: string helpers, field parsing, heap operations, command handling. The comparison reads only the two cached integers, so the ordering rule no longer

depends on the string format, which was what tied Mistral's comparison to its parser. Every function that can fail returns a status instead of failing silently. A full prologue and line-level comments were added to the standard the rubric requires, each block labeled with where it came from.